ARCHITECTURE-DECISIONS

Why OpenHIM (not just a reverse proxy), why JeMPI (not HAPI MDM / OpenCR), and every other trade-off in this stack

2026-08-28

- Architecture Decisions — Pros & Cons vs Alternatives
  - 1. Interoperability layer: OpenHIM — vs a plain reverse proxy / API gateway
  - 2. Patient matching / MPI: **external JeMPI** — vs HAPI FHIR MDM — vs OpenCR — vs none
  - 3. Health ID: mediator-minted Damm-checked random — vs alternatives
  - 4. Write-path matching: synchronous deterministic `crRegister` — vs async probabilistic — vs `/upload`
  - 5. Duplicate review: custom `admin-mediator` — vs JeMPI native — vs manual
  - 6. Clinical store: HAPI FHIR — vs alternatives
  - 7. Auth: Keycloak + `private_key_jwt` + **offline** verification — vs alternatives
  - 8. Reverse proxy / TLS: **Caddy** (VPS only) — vs nginx+certbot / Traefik / cloud LB
  - 9. JeMPI transport: WireGuard tunnel — vs public port + allowlist — vs mTLS — vs co-location
  - 10. Other choices (briefer)
  - 11. Summary scorecard

Architecture Decisions — Pros & Cons vs Alternatives

Each section: **what we chose**, **why**, **pros**, **cons / risks**, and the **alternatives** with how they compare. Everything is grounded in what is actually in this repo.

Legend: advantage · cost/risk · neutral/context-dependent.

---

1. Interoperability layer: OpenHIM — vs a plain reverse proxy / API gateway

What we chose

OpenHIM Core as the single front door for all client traffic. It authenticates the caller (Keycloak JWT or Basic), routes by **channel** (URL regex → mediator, gated by a **role**), records every request and every downstream call it triggers as a **transaction with an orchestration trail**, and returns an unwrapped body to the client. Source: `docker-compose.yml` (`openhim-core`), every mediator's `src/openhim.js`, `DEMO.md` §6.

Why

The system is an **OpenHIE** reference implementation. OpenHIM is the OpenHIE "Interoperability Layer" component. The value is not proxying — it is the **audit-grade transaction log** (an

irreversible patient merge, a JeMPI registration, a HAPI write are all captured with request/response bodies), plus **role-scoped channels** and **mediator self-registration**.

Pros

- **Every downstream call is logged with payloads** — the orchestration trail on a transaction is the audit record for identity-changing operations (`admin-mediator/src/openhim.js` comment: merges are routed through OpenHIM specifically so they land in the log).
- **Central auth + RBAC** — one JWT verification point, one role model (`np-fhir-clients`, `np-admin-clients`, `patient-portal-clients`, `np-fhir-intake-clients`); a portal credential physically cannot reach the write channel.
- **Mediators self-register** their channels on boot (`syncChannels()`) — no manual gateway config drift.
- **OpenHIE-aligned** — interoperable with other OpenHIE tooling, familiar to ministry-of-health integrators, a known reference architecture.
- **Response-envelope contract** lets a mediator return partial success + the full sub-call trace in one place.

Cons / risks

- **Heavier than a proxy** — MongoDB dependency, a Node service, its own Console, a self-signed cert story. More to run, patch and back up.
- `NODE_ENV=development` in the shipped compose; OpenHIM Core is `:latest` (unpinned).
- **Self-signed transaction/API certs** — clients need `-k` locally; real TLS only arrives via Caddy on a VPS.
- **Transaction log growth** — Mongo `transactions` collection is unbounded by default; retention must be configured.
- **Mongo 4.0** is old and unpinnable to arm64.

Alternatives

Alternative	vs OpenHIM here
Plain reverse proxy (nginx / Caddy / HAProxy)	tiny, fast, stateless. no transaction log, no per-request orchestration capture, no role→route model beyond basic `allow`/`deny`, no mediator registration. You would rebuild the audit trail yourself in each mediator. Fine for pure passthrough; **loses the main reason this stack exists** (auditable orchestration of identity changes).
API gateway (Kong / Tyk / APISIX)	mature rate-limiting, plugins, k8s-native, good observability (metrics/traces). still no payload-level transaction log by default; JWT/RBAC is comparable; you lose OpenHIE alignment and the mediator SDK; request/response body logging is a plugin you must secure (PHI at rest). A reasonable swap if OpenHIE compliance is not required and you add your own audit store.
Mirth Connect / Rhapsody (integration engine)	very strong message routing/transformation, channel history. HL7v2-centric heritage, heavier licensing (Rhapsody), less FHIR-native, not OpenHIE.
No layer — clients hit HAPI directly	simplest. HAPI has no auth in this deployment; no facility attribution; no JeMPI step; no audit. Not viable here.
OpenHIM + a gateway in front (belt & braces)	Some deployments put Kong/nginx in front of OpenHIM for rate-limiting + TLS and keep OpenHIM for orchestration/audit. This repo uses **Caddy** for exactly the TLS slice (VPS only) and keeps OpenHIM for everything else.

Verdict: OpenHIM earns its weight because the domain is patient-identity orchestration where an auditable, payload-level trail of every merge/register/write is a requirement, not a nice-to-have. If this were stateless FHIR passthrough, a reverse proxy or gateway would be the right call.

---

2. Patient matching / MPI: external JeMPI — vs HAPI FHIR MDM — vs OpenCR — vs none

What we chose

A **separate JeMPI** instance (own host, Dgraph-backed) called synchronously on every Patient write via `POST /crRegister` (deterministic find-or-create), with `POST /crCandidates` to resolve the golden UID on a duplicate. The mediator mints the human-facing Health ID and stores it as JeMPI's `auxId`. Source: `mediator/src/providers/jempiProvider.js`, `jempi-config/`, `FHIR_MEDIATOR_IMPLEMENTATION.md` §0.3.

Why

Cross-source identity that survives demographic variation and multiple hospitals, with a **tunable probabilistic engine** and a graph store built for entity resolution — kept **out of** the FHIR server so identity logic and clinical storage evolve independently (`hapi-config/application.yaml`: "Patient identity/matching moves to JeMPI").

Comparison

Dimension	JeMPI (chosen)	HAPI FHIR MDM	OpenCR
Nature	Standalone MPI service (Dgraph + Kafka + linker), OpenHIE "Client Registry"	A module inside the HAPI FHIR server	Standalone OpenHIE Client Registry (Hearth/ Elasticsearch, Matchbox-style rules)
Deployment cost	Heaviest — Dgraph alpha/ zero + Postgres + Kafka + Zookeeper + several services; its own VPS	Lightest — no new service, rides HAPI + its Postgres	Medium — Node + Elasticsearch
Matching model	Deterministic rules **+** Fellegi-Sunter probabilistic with per-field m/u weights; blocking queries; async linker with threshold band	Rule-based `mdm-rules.json` (matchers + weighted score, threshold); no true F-S m/u; deterministic-ish	Decision-rule / probabilistic via configurable matching; FHIR-native
FHIR-native	No — speaks its own JSON (`demographicData`, `sourceId`, `auxId`); the mediator maps FHIR↔JeMPI	Yes — operates on `Patient` resources directly, emits `Person` + `link`	Yes — FHIR `Patient`/`Person` in and out
Golden record	Explicit golden record + interactions in a graph; `auxId` is a first-class handle	HAPI `Person` resource with `link[].target` → source Patients	FHIR `Person` linking source `Patient`s
Review / stewardship	Native notification queue + `/relink` + audit trail (but **empty on our deterministic write path** —	`$mdm-query-links`, `$mdm-update-link`, `$mdm-merge-golden-resource`,	Match review via its own UI/API

Dimension	JeMPI (chosen)	HAPI FHIR MDM	OpenCR
	we built `admin-mediator` for it)	`$mdm-not-duplicate` — a usable steward API out of the box	
Scale	Graph DB + Kafka designed for large registries; horizontal-ish	Matching runs in the FHIR server's JVM/DB — couples MPI load to clinical query load	Elasticsearch scales; smaller community
Operational maturity	Younger project, fast-moving; schema changes need Dgraph regen (`jempi-config/README.md`)	Same lifecycle as HAPI, huge community, well documented	Smaller community, less active than JeMPI lately
Coupling	Identity and clinical storage fully decoupled — swap either side	MPI is welded to the FHIR server version and DB	Decoupled
This repo's fit	Chosen; national-scale ministry context, want a dedicated tunable engine	Rejected: would couple matching to HAPI and cap tuning; also `client_id_strategy: ANY` + our conditional-update pattern already fills the "one Patient per HID" need without MDM	Not used; JeMPI is the OpenHIE direction the wider project took (memory: "mediators moving from HAPI MDM to external JeMPI")

JeMPI pros (as used here)

- **Tunable probabilistic matching** with real m/u weights — and the `admin-mediator` **reuses the exact same weights file** for its own scoring, so "a score here means what a score there would mean".
- **Deterministic synchronous path** (`crRegister`) gives the write a golden UID in one call — the mediator can mint the HID and `PUT` to HAPI in the same request.
- **Decoupled** — HAPI can be upgraded/replaced without touching identity; JeMPI can be retuned without redeploying HAPI.
- **Graph store** purpose-built for entity resolution; `gidsAll` / `expandedGoldenRecords` / `customSearch/golden` give the review scanner what it needs.

JeMPI cons / risks (as observed)

- **Heavy to operate** — separate VPS, Dgraph schema regeneration on config changes (the `dob` index change is a schema change, not a restart — `jempi-config/README.md`).
- **Not FHIR-native** — a FHIR↔JeMPI mapping layer lives in the mediator and is a source of past bugs (feeding the Health ID into `nationalId` silently degraded matching to demographics-only).
- **Native review queue is unusable on our write path** — `crRegister` never scores or notifies, so JeMPI's `/notifications` is permanently empty. We had to build `admin-mediator` (scan + score + `/relink` with a fabricated `notificationId` that JeMPI does not validate).
- **Transport** — `:50000` has no TLS/auth; we rely on WireGuard or an IP allowlist. `:50001` (Keycloak-protected) is the production target but **not yet wired**.
- **`crRegister` is deterministic-only in practice** — the probabilistic hold band only runs on the async Kafka path, so "true" probabilistic auto-linking is not on the synchronous write path.
- **No split** — merges are effectively irreversible (this drove the stricter admin scoring).

If we had chosen HAPI FHIR MDM instead

- One fewer service, one fewer VPS, one fewer backup target, no tunnel.
- Native FHIR, native steward operations, native `Person/link`.
- Matching load competes with clinical query load in the same JVM/DB.
- Tuning is `mdm-rules.json` — capable, but not Fellegi-Sunter m/u; less suited to a national registry where you want to defend the score model.
- Harder to later swap the FHIR server.

If we had chosen OpenCR instead

- FHIR-native, decoupled, OpenHIE Client Registry.
- Smaller / less active community than JeMPI; Elasticsearch operational surface; the wider project already committed to JeMPI.

Verdict: JeMPI is the right call for a national, tunable, auditable registry that must stay decoupled from clinical storage — at the cost of being the heaviest and least FHIR-native option, and needing the `admin-mediator` to paper over its empty native review queue on a deterministic write path. For a single-jurisdiction or lower-scale deployment, HAPI MDM would be the pragmatic choice.

---

3. Health ID: mediator-minted Damm-checked random — vs alternatives

What we chose

10 random digits (first 1–9) + 1 **Damm** check digit, minted by `mediator/src/fhir/healthId.js`, stored as JeMPI's `auxId`, and used as the key for the HAPI conditional upsert. Opaque — encodes nothing.

Pros

- **Privacy by design** — leaking the number reveals no DOB/gender/facility.
- **Damm** catches every single-digit typo and every adjacent transposition (incl. twin-digit swaps Luhn misses) with one table, **no DB lookup** — a mistyped ID is rejected client-side.
- **Decoupled from JeMPI's UID** — human-facing card number vs machine dedup key, mapped 1:1; either representation can change independently.
- **Reused on re-registration** via JeMPI `auxId` (a consistent Dgraph read), so the same person always gets the same card number.

Cons / risks

- **`Math.random()`, not a CSPRNG** — opaque and check-digited, but not cryptographically unpredictable. `crypto.randomInt` would fix it.
- **Uniqueness is best-effort** — 5 HAPI existence checks + the conditional `PUT` as the real guard. Fine at 10^{10} space, but not a hard DB constraint.
- **Two identifiers to keep 1:1** — the invariant is code-maintained, not store-enforced (the admin merge has explicit rules to not break it).

Alternatives

Alternative	Trade-off
Use JeMPI's golden UID as the HID	one identifier. exposes an internal Dgraph id as a lifelong card number; changes if JeMPI is rebuilt; not human-friendly; couples the card number to the MPI implementation.
Use HAPI's resource UUID as the HID	zero minting. not human-transcribable, no check digit, changes on merge, couples the card number to the FHIR server.
Use the national NID as the HID	no new identifier. not everyone has one; privacy (NID is sensitive and reused across systems); this design deliberately keeps NID as a matching input, not the card number.
Sequential / block-allocated IDs (Luhn, like some MCI schemes)	compact, allocatable per facility. leaks volume/ordering; needs a central allocator (SPOF); Luhn misses twin-digit swaps.
Alphanumeric (planned)	A design exists (`docs/superpowers/plans/2026-08-28-...`) for `AA481242` + Damm over an internal letter→digit map. **Not implemented.** Would shorten the visible string; adds a mapping step.

Verdict: opaque + Damm + 1:1-to-golden-UID is a solid privacy-first choice; the only real weakness is `Math.random()`.

---

4. Write-path matching: synchronous deterministic `crRegister` — vs async probabilistic — vs `/upload`

What we chose

Every Patient write calls `crRegister` (synchronous find-or-create on the deterministic rules), so the golden UID is known in the same request and the HID can be minted and the HAPI `PUT` done in one round trip.

Pros

- **One request, fully resolved** — the caller gets a persisted Patient with both identifiers back immediately.
- **Predictable** — deterministic rules (`national_id`, or name+family+dob+phone+gender) are explainable to a data steward.
- **Placeholder-phone-safe** — the strict rule was designed around the `9800000000` placeholder being common.

Cons / risks

- **No synchronous probabilistic linking** — near-misses become separate golden records; the `admin-mediator` scan is the safety net, not the primary path.
- **JeMPI's native review queue stays empty** — no notifications are emitted.
- A same-person record entered once with a real phone and once with the placeholder won't match deterministically (falls through to probabilistic, which this path doesn't run synchronously).

Alternatives

Alternative	Trade-off
Async probabilistic path (`/upload` CSV → Kafka → linker)	real F-S auto-linking + threshold band + native review queue. **destroys the synchronous `goldenUID`** the write path needs to mint the HID and `PUT` in one request (`admin-mediator/README.md` explicitly rejects this). Would require a two-phase write (register now, reconcile later).
Deterministic + inline probabilistic call before persist	possible if JeMPI exposed a synchronous scored match; `/calculateScores` is **404** on this deployment, so not available.

Verdict: synchronous deterministic is the only option that keeps the "one request → persisted Patient + HID" contract on this JeMPI deployment. The `admin-mediator` closes the probabilistic gap out-of-band.

---

5. Duplicate review: custom `admin-mediator` — vs JeMPI native — vs manual

What we chose

A dedicated mediator that: scans golden records via `customSearch/golden`, scores pairs with the **deployed JeMPI m/u weights**, queues them in its own Postgres table, and merges a confirmed pair in **JeMPI then HAPI** (`/relink` with a self-generated `notificationId`).

Pros

- **Works despite the empty native queue** — three verified JeMPI quirks make it possible (`/relink` ignores `notificationId`; `customSearch/golden` finds near-misses `crCandidates` can't; `/calculateScores` is 404 so we score ourselves).
- **Merge lands in both stores atomically-ish** — JeMPI first (identity source of truth), then idempotent HAPI reconcile; a half-merge is a visible, retryable `MERGE_PARTIAL`.
- **Stricter than stock JeMPI where it matters** — exact match on `dob`/`national_id`/`phone_number` (JW on a date is nonsense; a near-miss national_id would saturate the score); placeholder phone treated as blank.
- **Auditable** — routed through OpenHIM so every merge is in the transaction log; `resolved_by` recorded.

Cons / risks

- **Relies on undocumented JeMPI behaviour** (`/relink` not validating `notificationId`) — could break on a JeMPI upgrade.
- **Merge without split** — irreversible; the strict scoring is the mitigation.
- **HAPI + JeMPI can drift** on a partial merge or a rollback — no automated reconciler.
- **The review UI is not in this repo** (`admin_portal`).
- Its own Postgres DB (`admin_review`) is another backup target; resolved decisions are **not** recoverable from JeMPI/HAPI.

Alternatives

Alternative	Trade-off
JeMPI native notification queue + steward UI	supported, no custom code. **empty** on our deterministic write path — would require moving to the async path (see §4) and giving up the synchronous HID mint.
HAPI MDM steward operations (`$mdm-query-links`, `$mdm-merge-golden-resource`)	mature, FHIR-native, no custom queue. only available if the MPI is HAPI MDM (see §2).
Manual SQL / spreadsheets	no audit, no concurrency guard, error-prone; not viable at scale.

Verdict: given the deterministic write path, a custom review mediator is essentially forced. It is well-reasoned (weights reuse, JeMPI-first ordering, strict scoring) but carries upgrade risk from depending on JeMPI internals.

---

6. Clinical store: HAPI FHIR — vs alternatives

What we chose

`hapiproject/hapi:latest` on PostgreSQL 16, with the NP Core IG loaded but **validation off**, `server_id_strategy: uuid`, `client_id_strategy: ANY`.

Pros

- **De-facto reference FHIR server** — huge community, R4-complete, free.
- **Conditional update / conditional create** support is exactly what the identifier-keyed upsert pattern needs.
- **NP Core IG** loaded so profiles/valuesets/naming systems are known (validation can be switched on later).
- **REST-hook subscriptions** enabled.

Cons / risks

- `:latest` (unpinned) — a pull can trigger an **irreversible schema migration** on the `hapi` DB.
- **No auth** in this deployment — safety is entirely network isolation + loopback bind; the Tester UI is unauthenticated read/write (compose comment flags "REVERT after the demo").
- Matching load and clinical load share one JVM/Hikari pool (`maximum-pool-size: 10`) — but MPI load is offloaded to JeMPI, which mitigates this.
- Validation **off** means non-conformant resources can land.

Alternatives

Alternative	Trade-off
Firely Server / Aidbox	commercial support, strong performance, good tooling. licensing cost; migration effort; less "reference implementation" cachet for a ministry.

Alternative	Trade-off
Google Cloud Healthcare API / Azure Health Data Services	✅ fully managed, scalable, compliant hosting. ❌ vendor lock-in, data residency (national health data off-shore), cost, less control over IG loading and conditional semantics.
Custom store	❌ reimplementing FHIR search/conditional-update is a multi-year trap. Not sensible.

Verdict: HAPI is the obvious choice for a reference OpenHIE deployment; the real gaps are operational (pin the image, add auth/validation, lock the Tester UI) not architectural.

---

7. Auth: Keycloak + `private_key_jwt` + offline verification — vs alternatives

What we chose

Hospitals authenticate to Keycloak with `private_key_jwt` (client-credentials, RSA keypair), get an RS256 JWT, and present it to OpenHIM, which verifies it **offline** against `openhim-config/realm-public.pem` and maps `sub` → OpenHIM Client. A protocol mapper forces `sub` = the NHFR code. BFFs use OpenHIM Basic auth.

Pros

- ✅ **No shared secrets with hospitals** — each holds its own private key; compromise is isolated and revocable per client.
- ✅ **Offline verification** — OpenHIM makes **no runtime call to Keycloak**; Keycloak can be down and existing tokens still verify.
- ✅ **`sub` = NHFR code** makes facility attribution trivial and non-forgeable (combined with OpenHIM setting `X-OpenHIM-ClientID`).
- ✅ Standard OIDC — hospitals' EMRs already speak it.

Cons / risks

- ❌ **Key rotation is manual and coupled** — a realm rebuild rotates the RS256 key, and you must re-run `extract-realm-pubkey.sh` + force-recreate `openhim-core` or **every token fails**. This is the single most common operational footgun in the setup docs.
- ❌ **Offline verification means no revocation check** — a revoked/short-lived token is honoured until it expires (tokens are 60 s in the helper, but that is client-side).
- ❌ **Keycloak `start-dev` + H2** — `docker compose down` wipes the realm and every hospital client; production needs `KC_DB: postgres`.
- ❌ Bootstrap admin ships `admin`/`admin`.
- ❌ Hospital private keys sit on disk in `keycloak/keys/` until hand-off.

Alternatives

Alternative	Trade-off
OpenHIM Basic auth for everyone	✅ simplest. ❌ shared secret per client, no standard rotation, weaker for external hospitals; used here only for the two internal BFFs.
Mutual TLS (client certs)	

Alternative	Trade-off
	strong, no token expiry juggling, transport-level. cert lifecycle/PKI to run; harder for EMR vendors; no fine-grained claims.
API keys	trivial. no expiry, no claims, no standard revocation, poor for PHI.
OpenHIM "openid" mode (online introspection)	real-time revocation. every request depends on Keycloak being up and fast; more coupling; not what this stack configured.
Managed IdP (Auth0/Okta/ Cognito)	less to operate. cost, data-residency for a national system, less control over realm import/mappers.

Verdict: Keycloak + `private_key_jwt` + offline verify is the right security posture for external hospital clients; the price is the **manual, tightly-coupled key-rotation dance** and the fragile `start-dev`/H2 realm store (both fixable).

---

8. Reverse proxy / TLS: Caddy (VPS only) — vs nginx+certbot / Traefik / cloud LB

What we chose

`caddy:2-alpine` behind a Compose profile, terminating Let's Encrypt TLS on `:443` for four subdomains, proxying to Keycloak, OpenHIM tx (`:5001`), OpenHIM API (`:8080` self-signed, verify skipped) and the Console.

Pros

- **Automatic ACME** — obtains and renews certs on boot, no cron, no certbot.
- **Tiny config** — the whole `Caddyfile` is ~25 lines.
- **Profile-gated** — local dev is completely unaffected; `docker compose up` never starts it.
- Handles the self-signed internal hop to the OpenHIM API cleanly (`tls_insecure_skip_verify` on that route only).

Cons / risks

- **Compose still publishes `5000/8080/9000` on all interfaces** — Caddy does not remove them; the **host firewall is the only thing** keeping them off the internet. Easy to forget.
- **DNS must resolve before first boot** or ACME fails and Let's Encrypt rate-limits.
- The `vps-ip` profile is **plain HTTP** — credentials unprotected; test only.
- No rate-limiting / WAF configured.
- Two Caddy services both bind `:80` — you must run one profile at a time.

Alternatives

Alternative	Trade-off
nginx + certbot	ubiquitous, well-understood, more middleware ecosystem. manual cert renewal wiring, more verbose config, no auto-HTTPS.
Traefik	auto-ACME + Docker label discovery + middlewares (rate-limit, auth). more moving parts / config model; label-based routing is powerful but easy to misconfigure.

Alternative	Trade-off
Cloud load balancer (ALB / GCLB / Cloudflare)	managed TLS, DDoS protection, WAF, no host process. cost, vendor coupling, another config plane, data-path egress considerations for a national system.
No proxy (local dev)	exactly what non-VPS mode does — direct host ports, self-signed.

Verdict: Caddy is the lowest-effort correct choice for auto-TLS on a single VPS. The **firewall dependency** is the sharp edge — it should be treated as a load-bearing security control, not an afterthought.

---

9. JeMPI transport: WireGuard tunnel — vs public port + allowlist — vs mTLS — vs co-location

What we chose

Default: mediators reach JeMPI at `http://10.8.0.1:50000/JeMPI` over a WireGuard tunnel between the OpenHIE VPS and the JeMPI VPS. Fallback: public `http://<jempi-ip>:50000/JeMPI` with a firewall IP allowlist.

Pros (WireGuard)

- **Transport encryption + access control** for a service that has **none of its own** on `:50000`.
- Modern, fast, small; kernel-level; simple config.
- Lets `:50000` stay bound to the JeMPI host's private interface.

Cons / risks

- **Container→tunnel routing needs a manual `iptables MASQUERADE`** rule (the container reaches `10.8.0.1` through Docker NAT, which isn't routed into `wg0`). Non-obvious; a frequent "host works, container hangs" symptom.
- Rule + `ip_forward` must be **persisted** or a reboot breaks JeMPI connectivity.
- Another daemon + key pair + peer config to manage; a key change on either side breaks the tunnel.
- Still **plain HTTP inside the tunnel** — fine, but `:50001` (Keycloak) is the real end state and isn't wired.

Alternatives

Alternative	Trade-off
Public `:50000` + firewall IP allowlist	zero tunnel ops. traffic crosses the internet **unencrypted** (PHI); allowlist is the only control; explicitly "test data only" in the setup docs.
mTLS between mediator and JeMPI	encryption + mutual auth, no tunnel daemon. JeMPI would need to terminate TLS / a sidecar; cert lifecycle.
JeMPI's `:50001` (Keycloak-protected)	real auth, the intended production path. **not wired** into the mediators yet (`FHIR_MEDIATOR_IMPLEMENTATION.md` §9); still wants transport encryption.

Alternative	Trade-off
Co-locate JeMPI in the same compose / same host / same VPC	no tunnel, private Docker network, lowest latency. couples the two lifecycles; JeMPI's heavy stack competes for resources; loses the clean "JeMPI is someone else's service" boundary; a `docker compose down` risk blast radius grows.
Site-to-site IPsec / cloud VPC peering	managed, no per-host daemon. provider-specific; heavier to stand up than WireGuard.

Verdict: WireGuard is a pragmatic encryption+access layer for a service with no auth, at the cost of the Docker-NAT masquerade gotcha and reboot-persistence care. The proper end state is `:50001` + transport security.

---

10. Other choices (briefer)

Choice	Why	Main pro	Main con / alternative
Many small mediators (4) instead of one monolith	each owns one channel/role; failure & deploy isolation; the read-only portal can't write	least-privilege per URL space; independent restart	4× `npm install` / `.env` /registration; shared code copied not shared. Alt: one mediator with internal routing (simpler ops, weaker isolation).
Conditional update keyed on `identifier` instead of `PUT /Patient/{healthId}`	race-safe, keeps `Patient.id` an opaque UUID, HAPI 412s on a true dup	idempotent + safe under concurrency	needs `client_id_strategy: ANY` anyway; slightly more opaque than a direct id PUT. Alt: `PUT /Patient/{numericHID}` (simpler, but the HID becomes the resource id and merges get messier).
Reads bypass JeMPI	identity already resolved at write time; the HID is the key	fast reads, JeMPI not on the read critical path	a stale HAPI record is served as-is; acceptable because writes converge it. Alt: resolve every read through the MPI (correctness on merges, big latency + coupling cost).
`admin_review` DB inside the `hapi-db` container (separate database)	reuse an existing Postgres; created at boot by the app	no new container	shares HAPI's `admin` superuser; couples backup/restore; a HAPI DB incident takes the review queue with it. Alt: own Postgres instance / own least-priv user.
Docker Compose, single host	fastest to stand up; matches a reference/demo posture	one file, one host, `restart: unless-stopped` for reboot survival	no HA, no rolling deploy, no autoscale; every DB is a SPOF. Alt: Kubernetes / Swarm + managed Postgres/Mongo for production.
Facility from `X-OpenHIM-ClientID`, never the payload	OpenHIM sets it post-auth and strips inbound copies — not forgeable	closes "hospital A claims to be hospital B"	needs the `sub`→NHFR mapper (or the `FACILITY_REGISTRY_JSON` fallback). Alt: trust `managingOrganization` (spoofable) — rejected.
NP Core IG loaded, validation OFF	mediators don't yet emit fully conformant resources	IG known now; flip validation on later without a data migration	non-conformant resources can persist meanwhile. Alt: validation ON now (blocks writes until mediators conform) or no IG (no path to conformance).
`meta.tag` provenance	a golden record that later proves to be two	makes a future split	HAPI unions `meta.tag` on PUT, so a cross-facility dedup accumulates two

Choice	Why	Main pro	Main con / alternative
(`source-facility`, `jempi-interaction-uid`) stamped on every write	people can be split by reassigning records — merges are recoverable, splits are not	feasible for records written from now on	`source-facility` tags (cosmetic, known). Alt: a `Provenance` resource per write (heavier, more correct FHIR).
`node --watch` dev runtime in containers	fast iteration during build-out	edit-reload	not a production build; source bind-mounted; `admin-mediator` notes `--watch` is unreliable over the bind mount. Alt: `CMD npm start`, no bind mount, `NODE_ENV=production`.

---

11. Summary scorecard

Decision	Fit for this context (national, auditable, OpenHIE)	Biggest risk to manage
OpenHIM interoperability layer	**Strong** — audit trail is the point	log growth; unpinned image; dev mode
JeMPI external MPI	**Strong for scale/tuning/ decoupling**; heavy	Dgraph schema regen; empty native review queue; not FHIR-native; `:50000` unauth
Mediator-minted Damm HID	**Strong** — privacy + typo-safety	`Math.random()` not a CSPRNG
Synchronous deterministic `crRegister`	**Necessary** to keep one-request writes	no synchronous probabilistic linking
Custom `admin-mediator` review/merge	**Forced by the above**; well-reasoned	depends on JeMPI internals; merge without split
HAPI FHIR clinical store	**Obvious choice**	pin the image; add auth; lock the Tester UI
Keycloak + private_key_jwt + offline verify	**Right for external clients**	manual coupled key rotation; H2 realm store
Caddy auto-TLS (VPS)	**Lowest-effort correct**	firewall is the load-bearing control
WireGuard tunnel to JeMPI	**Pragmatic encryption for an unauth service**	Docker-NAT masquerade + reboot persistence
Compose single-host	**Fine for reference/demo**	no HA; every DB a SPOF